

Break the Student Vault

Problem Statement

A college has developed the application to store student records. During a security audit, the development team discovers that sensitive data may not be adequately protected. Your team has been hired as an .

- Identify at least three confidentiality vulnerabilities.
- Explain how an attacker could obtain the student records.
- Demonstrate the weakness using only the provided test environment.
- Recommend secure countermeasures.

CHALLENGE-1

1. Introduction

- A college application is used to store and manage student records such as student name, register number, marks, and other academic information.
- During a security audit, it is found that sensitive student data may not be adequately protected.
- The objective of this security audit is to identify confidentiality vulnerabilities, understand how an attacker could potentially access student records in the provided test environment, demonstrate the weaknesses safely, and recommend suitable security countermeasures.

2. Confidentiality Vulnerabilities

Vulnerability 1: Plaintext Storage

Student records may be stored directly in a text file or database without encryption.

Risk:

If an attacker obtains the storage file, they may be able to read sensitive student information directly.

Impact:

- Student names may be exposed.
- Register numbers may be exposed.
- Marks and academic information may be disclosed.

Vulnerability 2: Weak Authentication

If the application uses a weak password, simple key, or does not properly authenticate users, unauthorized users may access student records.

Risk:

An attacker could gain access to the application without being an authorized user.

Impact:

Confidential student information could be viewed or copied.

Vulnerability 3: Insufficient Access Control

The application may not properly restrict access to student records based on user roles.

For example, if every logged-in user can access all student records, there is no proper separation between administrators, staff, and ordinary users.

Risk:

A low-privileged user may access records that should only be available to authorized staff.

Impact:

This can lead to unauthorized disclosure of student information.

3. How an Attacker Could Obtain Student Records

- In a controlled test environment, an attacker could potentially obtain student records through the following weaknesses:

Method 1 – Accessing Unprotected Storage

- If the application stores records in an unencrypted file, an attacker who has access to the test environment could inspect the file and read the information.

Example:

student_records.txt

Name: Dharshitha

Register No: 192521093

Marks: 95

If the data is stored like this, encryption is not being used to protect confidentiality.

Method 2 – Unauthorized Application Access

- If authentication is weak, an attacker may attempt to access the application using an invalid or unauthorized account.
- In the test environment, the security team can verify whether the application correctly rejects unauthorized login attempts.

Method 3 – Excessive File Permissions

- If sensitive files are readable by unnecessary users or processes, an attacker with a lowprivileged account may be able to access the records.
- The security team should check whether only authorized application users have permission to read the student database.

4. Demonstration in the Test Environment

The security weaknesses should be demonstrated only against the college's provided test application/environment.

Test 1 – Check Storage Protection

1. Open the test application's data directory.
2. Locate the student record file.
3. Check whether sensitive information is stored in readable form.
4. Verify whether encryption is being used.
5. Record the result as a confidentiality finding.

Expected finding:

- Student records are readable in plaintext.
- Therefore, an attacker with file access could obtain confidential student information.

Test 2 – Check Authentication

1. Open the test application.
2. Use the authorized test account.
3. Verify that authentication is required.
4. Attempt access with an invalid test credential.
5. Confirm that unauthorized access is rejected.

Test 3 – Check Access Control

1. Log in using the provided low-privilege test account.
2. Attempt to access records that should require administrator privileges.
3. Verify whether the application properly denies access.

4. Document the result.

5. Recommended Countermeasures

1. Encrypt Sensitive Data

- Student records should be encrypted before storage.
- Use strong modern encryption such as AES-GCM rather than storing sensitive information in plaintext.

2. Implement Strong Authentication

The application should use:

- Strong passwords
 - Secure password hashing
 - Account lockout/rate limiting
 - Multi-factor authentication for administrators where appropriate
- ### 3. Implement

Role-Based Access Control

Create different roles such as:

Administrator



Full student-record access

Staff



Required academic records only

Student



Own records only

Users should only have access to the information required for their role.

4. Protect File and Database Permissions

- Sensitive files should only be accessible to the application and authorized administrators.
- Unnecessary users should not have read or write permissions.

5. Secure Backups

- Database backups should also be encrypted because attackers may target backup files instead of the main database.

6. Secure Key Management

- Encryption keys should not be hard-coded inside the application or stored in the same location as the encrypted data.
- Keys should be securely managed and access should be restricted.

7. Security Logging and Monitoring

The application should record important security events such as:

- Successful logins
- Failed login attempts
- Unauthorized access attempts
- Access to sensitive records

This helps detect suspicious activity.

